

Stack overflow to control flow

Follow-along: play the attacker, then the defender

Overview

A network appliance runs a small service that registers devices. It reads one request line, copies the device name out of it into a 64-byte array, and replies. The copy is a loop the program wrote itself, and its only stopping conditions are the length of the request and the line terminator; nothing compares the number of bytes copied against the size of the array. A name longer than 64 bytes therefore keeps writing past the end of the array and on over whatever the compiler placed above it in the same stack frame, including the address the function will return to. Overwrite that address and the caller decides where the program goes next. In Part 1 you measure the distance to it, aim it at a function that prints the appliance's recovery key, and read the key off the wire.

Part 2 changes no line of the program. You compile the same source five times with different compiler and linker flags, restart the service on each build, and send the identical request at it. Four flags are on the list every hardening guide gives: checked C library calls, a read-only relocation table, a position-independent build, and a stack canary. Three of the four leave the key coming back and one stops it, and working out which is which is the exercise. This lab covers one bug class, a stack write past the end of an array in the same frame as the return address, and one technique, redirecting the return into code the binary already contains. Overflows on the heap, format-string bugs, and chaining several return addresses into a sequence are all outside it, and so is defeating a canary or a randomised address space by first leaking a value out of the process.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

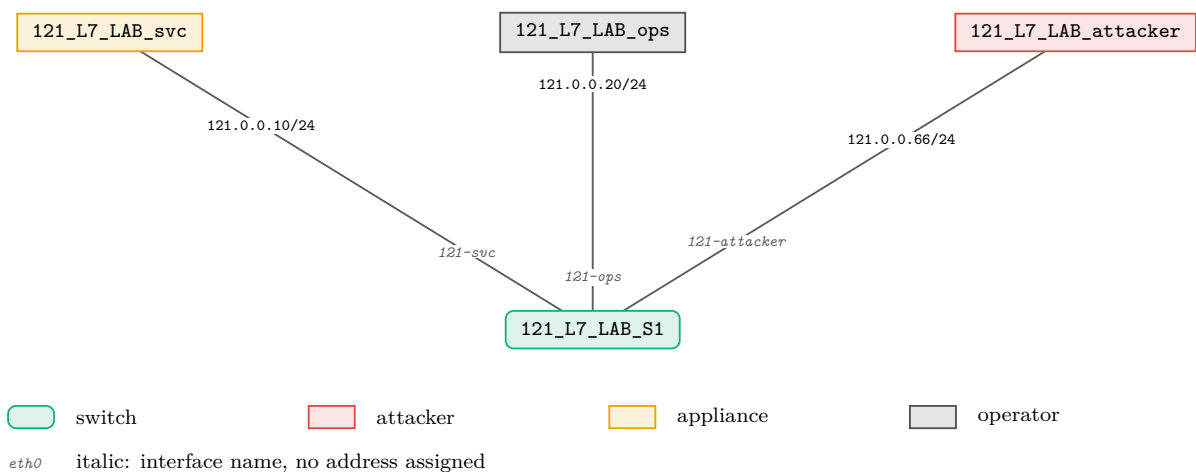
The root menu lists one category per row. Select **Software exploitation and containment**, then **Stack overflow to control flow**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle, so you read the before and after without shelling in. It also says which build the appliance is running and whether a request that overruns the name field still gets the recovery key back. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.

- **Shell command for a node** lists every node (svc, attacker, ops) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** puts the appliance back to the program and the build Part 1 starts from, and removes everything Part 2 compiled, without tearing the lab down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. You will want two terminals open for most of this lab: one on the appliance and one on the attacker's machine. **Back** (or Esc) returns to the lab list.

The setup



Three machines on one switched segment, with no router and nothing routed off the host.

Role	Container	Address	What it is
appliance	121_L7_LAB_svc	121.0.0.10	runs <code>devregd</code> on TCP 9000
operator	121_L7_LAB_ops	121.0.0.20	sends the one legitimate request
attacker	121_L7_LAB_attacker	121.0.0.66	sends every request in Part 1
switch	121_L7_LAB_S1	none	one Open vSwitch bridge

The appliance carries the tools you work with: `gcc` and `gdb`, and `nm`, `objdump` and `readelf` from binutils. The attacker's machine carries `python3` and `nc`. The source of the service is on the appliance at `/opt/devreg/src/devregd.c` and the builds live in `/opt/devreg/bin`. The source is mode 0444 and you never change it: every stage of Part 2 is a compiler flag, not an edit. Every shell in this lab is root inside its own container, so root can write the file anyway; the Reset action restores it, and a lab you have reset is a lab this handout still describes.

The service speaks one request. Send it a line and it replies on the same connection:

```
# 121_L7_LAB_ops: the operator's own machine, one request
printf 'REGISTER printer-07\n' | nc -q 2 121.0.0.10 9000
```

```
# 121_L7_LAB_ops prints this; it is output, not commands to type
registered: printer-07
```

Anything that is not a REGISTER line gets `usage: REGISTER <device-name>` back. That reply is the liveness check for the whole of Part 2: after every rebuild you send this same request and the same reply has to come back, because a build that stops the exploit by failing to run has defended nothing.

Assessment

There are 19 numbered questions in this handout, placed at the step each one belongs to. Answer them in a separate document, numbered to match. Several ask for a number or an address you read off your own screen; give the value you saw, not a value from someone else's run.

Part 1: the attack

1.1 Reading the bug

Open a shell on the appliance and read the program. Two functions matter and they are next to each other in the file.

```
# 121_L7_LAB_svc: the appliance
docker exec -it 121_L7_LAB_svc bash
```

```
# 121_L7_LAB_svc: read the copy and the function that calls it
sed -n '/^static void copy_field/,/^}/p' /opt/devreg/src/devregd.c
sed -n '/^static void register_device/,/^}/p' /opt/devreg/src/devregd.c
```

```
/* 121_L7_LAB_svc prints this; it is output, not code to type */
static void copy_field(char *dst, const char *src, size_t len)
{
    size_t i = 0;
    while (i < len && src[i] != '\n' && src[i] != '\r') {
        dst[i] = src[i];
        i++;
    }
    dst[i] = '\0';
}

static void register_device(const char *field, size_t len)
{
    char name[64];

    copy_field(name, field, len);
    printf("registered: %s\n", name);
    fflush(stdout);
}
```

`copy_field` is handed a pointer and a length, and the length is the length of what arrived on

the connection. It is not the size of `name`, and nothing in `copy_field` knows what that size is: a `char *` carries no size with it.

Definition: Stack frame, saved frame pointer, saved return address

Each call a program makes reserves a block of memory on the stack for that call: its local variables, and the two values the processor and the compiler need to get back afterwards. On x86-64, the `call` instruction pushes the address of the instruction to return to, and the called function's first instructions usually push the caller's frame pointer register (`rbp`) and set up its own. So above a function's locals sit the **saved frame pointer** and then the **saved return address**. The `ret` instruction at the end of the function pops that address off the stack and jumps to it. It does not check where it came from.

Definition: Buffer overflow

A write that goes past the end of an array and continues into whatever is next to it in memory. When the array is a local variable, what is next to it is the rest of that function's stack frame, and then the saved frame pointer and the saved return address. The array's size is a fact the compiler knows; a **buffer overflow** happens when the code doing the write does not use it.

Two things follow. The bytes the caller sends past 64 land somewhere, and the last of those places is the saved return address of `register_device`. And because the copy stops only at a newline, a carriage return, or the end of the request, a caller can send any byte value in between, including a zero byte, which matters once you are writing an address rather than a name.

1.2 A name that is too long

Open a second terminal on the attacker's machine and leave the appliance shell open in the first.

```
# 121_L7_LAB_attacker: the attacker's machine
docker exec -it 121_L7_LAB_attacker bash
```

Send a name of 70 characters, which is longer than the array:

```
# 121_L7_LAB_attacker
python3 -c 'import sys; sys.stdout.buffer.write(b"REGISTER " + b"A"*70 + b"\n")' | \
nc -q 2 121.0.0.10 9000
```

Question 1. Give the reply to the 70-byte name and the reply to the `printer-07` request from §The setup. The array holds 64 bytes and `copy_field` wrote 70 of yours plus a terminating zero byte into it. Say what the seven bytes past the end of the array landed on, and explain why the reply still came back given where `printf` sits in `register_device`.

1.3 Measuring the distance to the return address

Guessing how far past the array the return address sits does not work, because the compiler decides the layout of a frame and it does not have to put the array immediately below the saved values. Measure it instead.

The service reads its request from standard input, so running the program with no arguments makes it serve one request typed at a terminal instead of listening on a port. That is what lets

you run it under a debugger without a network connection in the way.

Definition: Standard input, standard output, standard error

Every process starts with three open file descriptors. Number 0, **standard input**, is where it reads; number 1, **standard output**, is where its results go; number 2, **standard error**, is where its diagnostics go. A shell redirects them: `< file` puts a file on standard input, `> file` sends standard output to a file, and `2> file` sends standard error to one. They are separate on purpose, so a program's output can be captured while its complaints still reach a person.

Write a pattern in which every four bytes are distinguishable, so that whichever four bytes land in the return slot say where in the pattern they came from:

```
# 121_L7_LAB_svc
python3 -c 'import sys, string; g = string.ascii_uppercase + \
    string.ascii_lowercase; sys.stdout.buffer.write(b"REGISTER " + \
    b"".join((c*4).encode() for c in g) + b"\n")' > /tmp/pattern
```

That writes AAAABBBBCCCC and so on for 52 groups, 208 bytes in all. Run the program under the debugger with that file on standard input.

Definition: Signal, SIGSEGV, SIGABRT

The kernel interrupts a process with a signal when something happens that the process has to be told about. **SIGSEGV** means the process referred to memory it may not touch, or to an address the processor rejects outright. **SIGABRT** is different in origin: it is the signal a process raises against itself through `abort()`, which is what the C library does when it detects that the program has already corrupted itself.

```
# 121_L7_LAB_svc
gdb -q /opt/devreg/bin/devregd
```

```
# 121_L7_LAB_svc: inside the gdb session you just opened
run < /tmp/pattern
x/s $rsp
```

The program prints its reply, then stops with **SIGSEGV**. On x86-64 with the 48-bit address layout Linux uses here, an address is only usable if its top 17 bits are all copies of bit 47; anything else the processor rejects rather than looks up, and eight bytes of printable ASCII never satisfy that. So `ret` faults on the address it popped instead of jumping to it, the fault leaves the stack pointer where it was, and `$rsp` still points at the eight bytes you supplied. `x/s $rsp` prints them.

Definition: Disassembly

An executable holds machine code, not source. **Disassembly** is the reverse of assembling it: reading those bytes back as instructions. `objdump -d` does it for a whole file and `gdb's disassemble` does it for one function. It is how you see what the compiler generated rather than what the source said, which is the only view that shows where a variable ended up.

Cross-check the number a second way, from the code the compiler generated. At `-O0` gcc

addresses a local array relative to the frame pointer, so the displacement in the instruction that passes `name` to `copy_field` is where the array starts:

```
# 121_L7_LAB_svc: inside the same gdb session
disassemble register_device
```

Question 2. Give the four-character group that `x/s $rsp` printed first, and convert it to a byte offset from the start of the name field. Then give the frame-pointer displacement the `lea` instruction in `register_device` uses for `name`, and show that the two numbers agree.

Question 3. Your offset is larger than 64. Name what sits between the end of the array and the saved return address in this build, give its size, and say what a compiler is free to do with it that would change your answer.

1.4 Choosing where to return

Definition: Symbol and symbol table

A compiler records the name and address of each function and global variable it emits in a **symbol table** inside the executable. `nm` prints it. The letter beside each entry says what kind it is: T is a function in the executable's own text, U is a name this file uses but does not define. A binary that has been through `strip` has no symbol table, and then the same addresses have to be found from the disassembly instead.

The program contains a function that prints the appliance's recovery key. A technician runs it from the serial console with `devregd --recovery`; no request routes to it and the service never calls it.

```
# 121_L7_LAB_svc
nm /opt/devreg/bin/devregd | grep ' T recovery_dump'
```

Definition: Load address

The address in the symbol table is where the function will be at run time only if the operating system loads the executable where the file says. That is the case for this build. Part 2 changes it.

Question 4. Give the address `nm` reports for `recovery_dump`. Write out the eight bytes you will put at the end of your request to make `ret` jump there, in the order they go on the wire, and say what fixes that order.

1.5 Sending it

The payload is the literal `REGISTER` and a space, then as many filler bytes as the offset you measured, then the eight bytes of the address. Nothing here can be typed as text: the address contains bytes with no printable form, and `python3` is on the attacker's machine to write them.

```
# 121_L7_LAB_attacker: put YOUR offset and YOUR address in
python3 -c 'import sys; sys.stdout.buffer.write(b"REGISTER " + b"A"*<OFFSET> + \
(0x<ADDRESS>).to_bytes(8, "little") + b"\n")' | nc -q 2 121.0.0.10 9000
```

Question 5. Quote both lines that came back. One of them the service printed on its way through `register_device` and one of them it printed after control left that function. Say which is which, name the instruction that transferred control, and explain how the processor came to be executing a function that nothing in the program calls and whose address appears nowhere in the program's own code.

Question 6. Predict, before running it, what comes back if you send four filler bytes fewer and leave the address bytes unchanged. Then send it and say whether your prediction held and why.

Run the Status action now. It reports which build the appliance is running, the offset it measured from that build, and whether a request that overruns the name field gets the key back. Everything in Part 2 is read from the same two result lines at the bottom of it.

Part 2: the defence

You cannot fix this by editing the program, because in this lab you do not own the program: the source is what the vendor shipped and the appliance runs what it was given. What you do own is how it is built. Every mainstream toolchain carries hardening options that can be turned on for an existing source tree with no change to a line of it, and what this part establishes is what each one actually defends.

You will compile the same file five times, restart the service on each build, and send the request from §1.5 at it again. After each one, check both facts: whether the key still comes back, and whether a legitimate `REGISTER printer-07` still gets its reply. A build that stops the exploit by failing to start has defended nothing, which is why the second check is not optional.

Two commands are used at every stage. Compiling:

```
# 121_L7_LAB_svc: the shape of every build in this part
gcc <FLAGS> -o /opt/devreg/bin/<NAME> /opt/devreg/src/devregd.c
```

And restarting the service on a build:

```
# 121_L7_LAB_svc
pkill -f /opt/devreg/bin
setsid setarch -R /opt/devreg/bin/<NAME> 9000 >>/var/log/devregd.log 2>&1 &
```

`setarch -R` turns address-space randomisation off for the process it starts and for its children, and for nothing else on the machine. It matters from §2.3 onwards; until then the addresses in these builds do not move either way. The service's own output goes to `/var/log/devregd.log`, and the service never puts standard error on the connection, so anything the C library says about the process is readable there and not by whoever sent the request.

2.1 Checked C library calls

Definition: `_FORTIFY_SOURCE`

A macro the C library's own headers read. With it defined and the compiler optimising, calls to a fixed list of library functions are replaced by checked versions with `_chk` on the end of the name: `memcpy` becomes `__memcpy_chk`, `printf` becomes `__printf_chk`, and so on. The checked version takes an extra argument, the size of the destination as the compiler computed it with `__builtin_object_size`, and aborts the process if the call would write past it. The list is a list of *library functions*. Code the program wrote itself is not on it.

```
# 121_L7_LAB_svc
gcc -O2 -D_FORTIFY_SOURCE=2 -fno-stack-protector -no-pie -o \
    /opt/devreg/bin/devregd-fortify /opt/devreg/src/devregd.c
nm /opt/devreg/bin/devregd-fortify | grep '_chk'
```

The macro is written together with `-O2` because the C library's headers put the checked declarations behind `__OPTIMIZE__`. Built at `-O0` the same command produces a binary with no `_chk` symbol in it at all, and a flag that generated nothing would tell you nothing.

Question 7. List the checked calls `nm` found in this build. Then look again at `copy_field` and say which library function the compiler would have had to see in it to catch this overflow, and why there is none for it to replace.

Restart the service on this build and send the request from §1.5 again, with the address re-read from *this* binary rather than the previous one.

Question 8. Before you send it: this build was compiled at `-O2` and the previous one at `-O0`. Predict whether the offset you measured in Question 2 still applies. Then measure it on this build the same way you measured it before, and give the number.

Question 9. Run `objdump -d /opt/devreg/bin/devregd-fortify` and look for `register_device` in it. Say what happened to that function, and name the function whose saved return address your request now overwrites instead. Then state whether the key came back, and whether REGISTER printer-07 still got its reply.

2.2 A read-only relocation table

Definition: Global offset table and RELRO

A dynamically linked program does not know at compile time where the library functions it calls will be. The linker gives it a table of addresses, the **global offset table**, which the dynamic loader fills in, and each call goes indirectly through an entry in it. An attacker who can write to that table can change where a call goes. **RELRO** (relocation read-only) is the linker making the table read-only once the loader has finished with it. `-z relro` marks the region; `-z now` makes the loader resolve every entry at start-up so the region can be sealed immediately rather than left writable for lazy resolution. Both together are what tools call full RELRO.

```
# 121_L7_LAB_svc
gcc -O0 -fno-stack-protector -no-pie -Wl,-z,relro,-z,now -o \
    /opt/devreg/bin/devregd-relro /opt/devreg/src/devregd.c
readelf -lW /opt/devreg/bin/devregd-relro | grep GNU_RELRO
readelf -dW /opt/devreg/bin/devregd-relro | grep -E 'BIND_NOW|NOW'
```

Run the same two commands against `/opt/devreg/bin/devregd`, the build from Part 1.

Question 10. One of those two lines is present in the Part 1 build as well and one is not. Say which, and state what the missing one adds.

Restart on this build, re-read the address, and send the request again.

Question 11. State whether the key came back, and whether `REGISTER printer-07` still got its reply. Then explain, from what the global offset table is for, why this flag was never going to change the outcome of this particular attack.

2.3 A position-independent build

Definition: Position-independent executable and load base

An ordinary executable names a fixed address for every function, and the loader must map it there. A **position-independent executable** is built so it works wherever it is put: every reference is relative, and the loader picks a **load base** and maps the file at it. `readelf -h` reports the file type as `EXEC` for the first kind and `DYN` for the second. In a `DYN` file, the address `nm` prints is an offset from the base, not an address.

Definition: Address-space layout randomisation

A kernel feature that draws the addresses a process is laid out at fresh on each `execve()` rather than using the same ones every time: the stack, the heap, the shared libraries, and, for a `DYN` executable, the executable itself. On Linux it is controlled by `kernel.randomize_va_space`, which is one setting for the whole machine. `setarch -R` is the per-process alternative: it sets the `ADDR_NO_RANDOMIZE` personality flag on one process, which the process's children inherit, and leaves every other process on the machine alone.

```
# 121_L7_LAB_svc
gcc -O0 -fno-stack-protector -fPIE -pie -o /opt/devreg/bin/devregd-pie \
    /opt/devreg/src/devregd.c
readelf -hW /opt/devreg/bin/devregd-pie | grep Type
nm /opt/devreg/bin/devregd-pie | grep ' T recovery_dump'
```

Restart on this build, with `setarch -R` as before, and send the request from §1.5 exactly as you sent it in Part 1, with the address unchanged.

Then find where the build was actually mapped. The service is a process on the appliance and its memory map is readable there:

```
# 121_L7_LAB_svc
head -1 /proc/$(pgrep -f /opt/devreg/bin)/maps
```

Question 12. Give the address `nm` printed, the load base from the memory map, and the sum. Send the request with the sum as the return address and state whether the key came back.

Now start the same build without `setarch -R`, so randomisation applies to it, and do it twice, reading the load base each time:

```
# 121_L7_LAB_svc
pkill -f /opt/devreg/bin
setsid /opt/devreg/bin/devregd-pie 9000 >>/var/log/devregd.log 2>&1 &
head -1 /proc/$(pgrep -f /opt/devreg/bin)/maps
```

Question 13. Give the two load bases you read. Send the request with the address computed from the first base against the second run, and state what came back. Then compute the address from the second base, send it again, and state what came back.

The second of those works because you read the base off the appliance, which is the machine you are defending rather than the machine you are attacking from. Someone with only a connection to TCP 9000 has no such read: they would have to make the process disclose an address, or guess one. That is a different exercise and this lab does not build it.

The service forks a child for each connection and the child inherits the parent's address space, so every connection to one running service sees the same base. The base changes when the service is started again, not when a new request arrives.

You have now seen a position-independent build with randomisation off, and one with it on. Do the remaining case: start the Part 1 build, which is not position independent, without `setarch -R`, and send the request from §1.5 with the address you used there.

```
# 121_L7_LAB_svc
pkill -f /opt/devreg/bin
setsid /opt/devreg/bin/devregd 9000 >>/var/log/devregd.log 2>&1 &
```

Question 14. You have four cases now: each of the two builds run with randomisation on and with it off. Give the result of each. Then state what the position-independent build contributes, what randomisation contributes, and why neither is a defence against this attack without the other.

2.4 A stack canary

Definition: Stack canary

A value the compiler stores in each protected frame between the local variables and the saved return address, and checks immediately before the function returns. The value is read from a per-process secret, so it cannot be predicted from the binary. If the check fails, the function does not return; it calls `__stack_chk_fail`, and in the GNU C library that prints `*** stack smashing detected ***: terminated` and aborts the process. `-fstack-protector` protects functions with a character array of a certain size; `-fstack-protector-strong` widens the set to any function with a local array or one whose address is taken.

```
# 121_L7_LAB_svc
gcc -O0 -fstack-protector-strong -no-pie -o /opt/devreg/bin/devregd-canary \
/opt/devreg/src/devregd.c
nm /opt/devreg/bin/devregd-canary | grep stack_chk
```

Restart on this build, re-read the address of `recovery_dump` from it, and send the request. Then look at the appliance's own log.

```
# 121_L7_LAB_svc
tail -5 /var/log/devregd.log
```

Question 15. Quote what the attacker's terminal received and what the log line says. Explain why the two are different, given which of the child process's three descriptors the service replaces with the connection.

Question 16. Measure the offset in this build the way you measured it in Question 2. Give the number and say how many bytes larger it is than the Part 1 build's. From the disassembly of `register_device`, account for those extra bytes: give the frame-pointer displacement of the canary and of `name`, and say what occupies the gap between them.

Question 17. Send the request again with a much longer run of filler, say 120 bytes, and the address at the end. State what came back. Then explain why no length of overflow gets past this check, using where the canary sits in the frame.

2.5 The build to ship

Compile the source once more with all four flags at once. This is the end state the lab grades.

```
# 121_L7_LAB_svc
gcc -O2 -D_FORTIFY_SOURCE=2 -fstack-protector-strong -fPIE -pie -Wl,-z,relro,-z,now \
-o /opt/devreg/bin/devregd-hardened /opt/devreg/src/devregd.c
```

Restart the service on it, send the request from §1.5 with both numbers recomputed for this build, the address and the offset, and send `REGISTER printer-07` from the operator's machine. Then run the Status action.

Question 18. Status prints four hardening properties and two result lines. Give all six for this build. Then name the one flag of the four that changed the result lines, and for each of the other three name the attack it does defend, which this lab did not perform.

Question 19. The canary stops the return address from being used. It does not stop it from being written: by the time the check runs, the copy has already finished. Name one thing the program has already done to its own memory at that point, and say what that means for a bug like this one in a function that never returns.

Checks: what “done” means

Run the Status action. Two lines at the bottom have to read `[done]` at once:

- **the exploit is dead:** the overflow no longer reaches `recovery_dump()`, so no recovery key comes back to 121.0.0.66.

- **the service still works:** a `REGISTER printer-07` from 121.0.0.20 still returns `registered: printer-07`.

Above those two lines `Status` prints the four hardening properties it read out of the running build. It reads them out of the file itself rather than out of the command you typed, so a flag that was written and had no effect shows there as having had none. Most of the builds you make in Part 2 leave the first of the two lines blank while that block reports hardening as present, and that is the result the lab is about.

The `Reset` action puts the appliance back where Part 1 started: the vulnerable build recompiled and running, everything Part 2 wrote removed, the source restored, and the log emptied.

Safety note

This lab is isolated. The three host containers are wired to one Open vSwitch bridge inside the switch container, with no address on the host and no route off it, so nothing you send reaches anything except the two other containers this lab spawned. The recovery key is a string compiled into this lab's own binary. The service, the overflow and every build you make exist only inside these containers and are removed with them.

Tear the lab down

When you have finished, remove the lab: pick **Teardown** from this lab's menu in the TUI, or run `scripts/teardown.sh` directly. It removes this lab's containers and nothing else.